AUTONOMOUS RECONNAISSANCE WITH AI-POWERED VISION AND PATH PLANNING

A PROJECT REPORT

Submitted by

VEERA SARAVANAN S	**22MT060**
JANISHA G	**22MT022**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

MECHATRONICS ENGINEERING

CHENNAI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)
(Affiliated to Anna University, Chennai)
CHENNAI-600 069

ANNA UNIVERSITY: CHENNAI-600 025

MARCH-2026

ANNA UNIVERSITY: CHENNAI-600 025

BONAFIDE CERTIFICATE

Certified that this project report "**AUTONOMOUS RECONNAISSANCE WITH AI-POWERED VISION AND PATH PLANNING"** is the bonafide work of **VEERA SARAVANAN S (22MT060), JANISHA G (22MT022)** who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
Dr.S.CHANDRA VANDHANA M.E,Ph.D.	Mr.J.Jai Ganesh M.Tech(Ph.D).
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Professor	Assistant Professor
Department of Mechatronics Engineering,	Department of Mechatronics Engineering,
Chennai Institute of Technology,	Chennai Institute of Technology,
Kundrathur,	Kundrathur,
Chennai-600069.	Chennai-600069.

Certified that the above students have attend of viva voce during the exam held on …….

INTERNAL EXAMINAR **EXTERNAL EXAMINAR**

ACKNOWLEDGEMENT

We express our gratitude to our Chairman **Shri. P. SRIRAM** and all trust members of Chennai institute of technology for providing the facility and opportunity to do this project as a part of our undergraduate course.

We are grateful to our Principal **Dr. A. RAMESH, M.E., Ph.D.,** for providing us the facility and encouragement during the course of our work.

We sincerely thank our Head of the Department , **Dr. S. Chandravadhana M.E., Ph.D.,** Department of Mechatronics Engineering for having provided us valuable guidance, resources and timely suggestions throughout our work.

We sincerely thank our Project Guide, **Mr. J. Jai Ganesh M.Tech(Ph.D).,** Professor, Department of Mechatronics Engineering for having provided us valuable guidance, resources and timely suggestions throughout our work.

We would like to extend our thanks to our **faculty coordinators of the Department of Mechatronics Engineering**, for their valuable suggestions throughout this project.

We wish to extend our sincere thanks to all **Faculty members of the Department of Mechatronics Engineering** for their valuable suggestions and their kind cooperation for the successful completion of our project.

We wish to acknowledge the help received from the **Lab Instructors of the Department of Mechatronics Engineering** and others for providing valuable suggestions and for the successful completion of the project.

VEERA SARAVANAN S

JANISHA G

ABSTRACT

The DogBot Recon System is an autonomous reconnaissance robot for indoor and outdoor navigation using real-time computer vision and AI-driven control. It integrates an ESP32-CAM for vision and motor control with a Python backend that handles image processing, obstacle detection, path planning, and autonomous movement. Multi-sensor fusion combines background subtraction, edge detection, optical flow, gradient analysis, vertical structure detection, and adaptive floor color detection, while object recognition and tracking use a lightweight YOLOv8n-seg model. A low-latency pipeline builds an occupancy grid, predicts obstacle motion with Kalman filtering, evaluates paths, and outputs steering and speed with emergency stop for nearby obstacles.

A dual-layer AI Decision Engine uses a fast local planner with a cloud fallback for low-confidence cases, supported by multi-frame voting for stability. Remote connectivity is enabled via secure MQTT, and a FastAPI dashboard with WebSocket streaming provides live video, telemetry, and manual or semi-autonomous control. The hardware includes an ESP32-CAM with OV2640 camera, L298N motor driver, four N20 gear motors, and a battery pack, achieving real-time navigation at about 15 FPS with low latency.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1.	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 SCOPE OF THE PROJECT	4
2.	**LITERATURE REVIEW**	**6**
	2.1 INTRODUCTION	6
	2.2 EVOLUTION OF RECONNISSANCE ROBOTICS	6
	2.3 REVIEW OF RELEVANT RESEARCH PAPERS	7
3.	**MATERIALS AND METHODS**	**16**
	3.1 SYSTEM ARCHITECTURE	16
	3.2 COMMUNICATION AND CONTROL STRATEGY	19
	3.3 CONFIGURATION MANAGEMENT APPROACH	21
	3.4 HARDWARE DESIGN AND IMPLEMENTATION	22
	3.5 ELECTRONIC COMPONENTS	24

	3.6 DC MOTORS, WHEELS AND LEG SYSTEM	27
	3.7 POWER SUPPLY	29
	3.8 CIRCUIT DIAGRAM & PIN MAPPING	29
	3.9 HARDWARE ASSEMBLY	31
	3.10 DEVELOPMENT ENVIRONMENT & TOOLS	32
	3.11 INTELLIGENT PROCESSING AND NAVIGATION	34
4	**RESULTS & TESTING**	**41**
	4.1 TESTING ENVIRONMENT SETUP	41
	4.2 OBSTACLE DETECTION PERFORMANCE	42
	4.3 PATH PLANNING PERFORMANCE	43
	4.4 REAL-TIME FPS & LATENCY MEASUREMENTS	44
	4.5 MQTT REMOTE CONNECTIVITY TESTING	45
	4.6 DASHBOARD DEMONSTRATION	46
5	**CONCLUSION & FUTURE SCOPE**	**48**
	5.1 CONCLUSIONS	48
	5.2 FUTURE ENHANCEMENTS	49
	REFERENCES	**51**
	PO & PSO ATTAINMENT	**54**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.3.1	Configuration System Sections	22
3.5.1	ESP32-CAM Module Specifications	25
3.5.2	L293D Motor Driver Specifications	26
3.10.1	Software Dependencies & Tools	32
4.4.1	System Performance Summary	44
4.5.1	MQTT Remote Connectivity Test Results	45

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1.1	Dog Bot Recon System Design	5
2.3.2	Yolo v8 Architecture	10
2.3.3	Occupancy Grid Representation	12
3.1.1	System Block Diagram	17
3.1.2	Frame Processing Pipeline Flowchart	18
3.2.1	Communication Architecture	19
3.2.2	Manual & Semi-Auto Mode	20
3.4.1	Solidworks CAD Model	23
3.4.2	3D-Printed Chassis Parts	24
3.5.1	ESP32-CAM Module	26
3.5.2	L293D Motor Driver	27
3.6.1	N20 Motor DC Motor with wheels	28
3.8.1	Circuit Diagram	30
3.8.2	Motor Direction Truth Table	30
3.9.1	Hardware Assembly	31
3.11.1	AI Decision Engine Dual-Layer Architecture	38
4.1.1	Assembled Hardware	41
4.2.1	Object Detection Accuracy by Distance range	42
4.2.2	Object Segmentation & Tracking	43
4.3.1	Path Planner Performance Metrics	44
4.6.1	Dashboard Demonstration	46

CHAPTER – 1

INTRODUCTION

1.1 BACKGROUND

The field of autonomous mobile robotics has witnessed remarkable growth over the past decade, driven by advances in embedded computing, computer vision, machine learning, and wireless communication technologies. Autonomous robots are increasingly being deployed in applications ranging from warehouse logistics and agricultural monitoring to search and rescue operations, environmental surveillance, and military reconnaissance. These systems must perceive their environment, make intelligent decisions, and navigate safely without continuous human intervention.

The emergence of low-cost, high-performance microcontrollers such as the ESP32 series has democratized robotics development. The ESP32-CAM module, in particular, combines a dual-core 240 MHz processor, built-in Wi-Fi and Bluetooth, and a 2-megapixel camera in a compact, affordable package. This makes it possible to build vision-capable robots at a fraction of the cost of traditional platforms. However, the limited computational power of such microcontrollers necessitates offloading intensive processing tasks such as deep learning inference and path planning to an external backend server.

Simultaneously, breakthroughs in real-time object detection models like YOLO (You Only Look Once), now in its eighth generation (YOLOv8), have made it possible to detect, classify, and segment objects in video streams at high frame rates even on CPUs without dedicated GPU

hardware. When combined with classical computer vision techniques such as background subtraction, optical flow, and edge detection, these deep learning models provide robust multi-modal perception capabilities.

The motivation behind the Dog Bot Recon System stems from the need for an affordable, modular, and intelligent reconnaissance robot that can autonomously explore unknown environments, detect and avoid obstacles in real time, and be controlled remotely from anywhere in the world via the internet. The project aims to demonstrate that sophisticated autonomous navigation can be achieved by combining inexpensive hardware with modern software engineering practices, real-time computer vision, and AI-powered decision making.

1.2 PROBLEM STATEMENT

Existing autonomous robot platforms suffer from several limitations that restrict their accessibility and practical deployment:

- **High Cost:** Commercial autonomous platforms such as Boston Dynamics Spot, Clearpath Robotics Jackal, or TurtleBot3 cost thousands of dollars, making them inaccessible for educational institutions and individual researchers.
- **Limited Remote Access:** Most hobby-grade robots operate only on local networks, preventing remote monitoring and control over the internet from distant locations.
- **Single-Method Detection:** Many low-cost robots rely on a single obstacle detection method (typically ultrasonic sensors), which provides limited spatial resolution and cannot identify object types.

- **No Intelligent Path Planning:** Budget robots often use simple reactive behaviors (stop and turn) rather than predictive path planning that considers obstacle velocities and future positions.
- **Fragmented Software:** The software for such systems is often monolithic and difficult to extend, lacking proper service-oriented architecture for modularity.

This project builds a modular, cost-effective autonomous robot with vision-based obstacle detection, deep learning recognition, predictive path planning using Kalman filtering, AI decision-making, and MQTT-based remote connectivity.

1.3 OBJECTIVES

The primary objectives of the Dog Bot Recon System project are:

- To design and build an autonomous reconnaissance robot using the ESP32-CAM microcontroller as the hardware platform with an L298N motor driver for differential drive control.
- To develop a multi-sensor fusion computer vision pipeline that combines six independent detection methods (background subtraction, edge detection, gradient magnitude, optical flow, thin edge detection, and adaptive floor color detection) for robust obstacle identification.
- To integrate YOLOv8n-seg deep learning model for real-time object detection and instance segmentation with persistent multi-object tracking using the Byte Track algorithm.
- To implement a five-stage real-time path planning engine consisting of occupancy grid construction, Kalman filter-based

obstacle state estimation, cost map generation, trajectory evaluation with multi-objective scoring, and temporal smoothing for stable navigation.

- To build an AI Decision Engine with dual-layer architecture: fast local path planner as primary, with cloud-based LLM fallback for complex scenarios, and multi-frame voting for noise-resistant direction decisions.
- To establish worldwide remote connectivity using MQTT protocol over TLS-encrypted connections, enabling motor control and status monitoring from any internet-connected device.
- To create a real-time web dashboard with live video streaming, telemetry display, manual and semi-automatic control modes, and automotive-style visualization using Fast API and WebSocket protocols.
- To achieve real-time performance targets of 15 FPS processing rate, sub-2ms path planning latency, and sub-150ms end-to-end local latency.

1.4 SCOPE OF THE PROJECT

The scope of the Dog Bot Recon System encompasses the complete design, development, and testing of an autonomous reconnaissance robot platform. The project covers the following areas:

- **Hardware Scope:** The hardware design focuses on integrating the AI-Thinker ESP32-CAM, L298N motor driver, four N20 DC gear motors, chassis, and 4×AA battery supply, including circuit connections and pin mapping.

- **Software Scope:** The software development spans three major areas. First, the ESP32 firmware written in Arduino C++ handles camera streaming, motor control, WiFi connectivity, and MQTT communication. Second, the Python backend built with FastAPI implements the complete vision processing pipeline, machine learning detection, path planning, AI decision making, and WebSocket-based communication. Third, the web dashboard provides the user interface for monitoring and control.

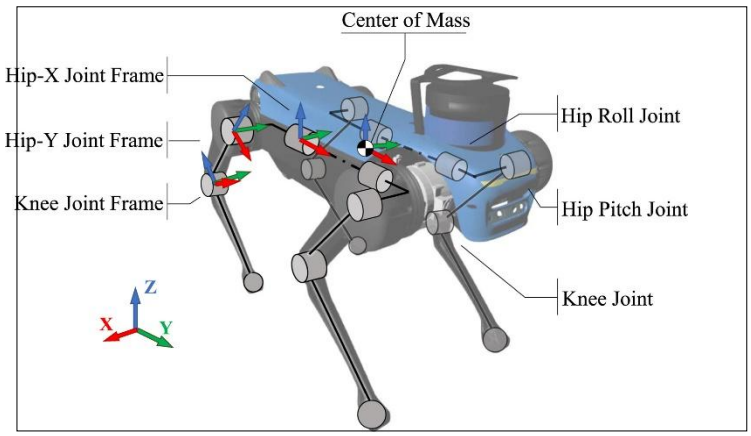

fig: 1.1.1 **Dog Bot Recon System Design**

- **Connectivity Scope:** Remote connectivity through MQTT protocol with TLS encryption enables worldwide access. The system supports both local HTTP communication and remote MQTT communication with automatic failover between modes.

CHAPTER – 2

LITERATURE SURVEY

2.1. INTRODUCTION

The field of autonomous robotic reconnaissance has experienced rapid advancement driven by improvements in sensor technology, artificial intelligence, and wireless communication systems. Research in this domain spans multiple disciplines including robotics, computer vision, machine learning, embedded systems, and IoT communication protocols. Understanding the current state of technology and identifying gaps in existing solutions forms the foundation for developing an effective reconnaissance robotics system such as the Dog Bot.

Contemporary robotic platforms have demonstrated increasing sophistication in navigation, environmental adaptation, and autonomous decision-making. The integration of multiple sensor modalities with advanced processing capabilities has enabled robots to perform complex tasks in challenging environments previously accessible only to human operators. In particular, the convergence of lightweight deep-learning models, low-cost microcontrollers with integrated cameras, and cloud-based large language models has opened new possibilities for building affordable yet capable autonomous systems.

2.2. EVOLUTION OF RECONNISSANCE ROBOTICS

Early robotic surveillance systems were mainly remote-controlled vehicles with basic cameras, requiring continuous human monitoring and operating only in controlled environments. The introduction of

autonomous navigation reduced human dependence and improved operational flexibility.

In the early 2000s, probabilistic robotics improved robot reliability through methods such as Kalman and particle filtering, enabling accurate localization and mapping under sensor uncertainty. After 2012, deep learning enabled real-time object detection and shifted surveillance robots toward intelligent perception.

Recent advancements in low-cost embedded hardware such as ESP32-CAM have made robotic surveillance more affordable and accessible. Modern systems now combine machine learning for object recognition with emerging LLM-based reasoning to support smarter decision-making, faster responses, and reduced manual supervision.

2.3. REVIEW OF RELEVANT RESEARCH PAPERS

Paper 1: "Probabilistic Robotics" by Thrun, S., Burgard, W., and Fox, D. (2005)

Thrun, Burgard, and Fox authored the seminal textbook "Probabilistic Robotics," published by MIT Press, which established the theoretical foundation for modern autonomous mobile robots. The book presents probabilistic frameworks including Bayesian filters, particle filters, and extended Kalman filters for the core problems of localization, mapping, and Simultaneous Localization and Mapping. Their central thesis is that probabilistic approaches can gracefully handle the inherent uncertainty in sensor measurements and robot motion, enabling reliable autonomous navigation in real-world environments where deterministic methods fail.

The textbook provides detailed mathematical derivations and practical

algorithms for problems ranging from grid-based localization and Monte Carlo localization to graph-based SLAM and FastSLAM. Each algorithm is grounded in probability theory, giving practitioners a principled way to reason about uncertainty in perception and action. The work has become one of the most cited references in robotics, with over 15,000 citations, and remains the standard graduate-level textbook on the subject.

In the context of the DogBot system, the probabilistic reasoning principles from this work inform the obstacle state estimation module, which maintains belief distributions over obstacle positions and velocities. Although the Dog Bot does not perform full SLAM (it uses reactive navigation rather than map-building), the Bayesian update framework from Thrun et al. underpins the Kalman filter–based obstacle tracker that enables predictive collision avoidance.

Paper 2: "A Formal Basis for the Heuristic Determination of Minimum Cost Paths" by Hart, P. E., Nilsson, N. J., and Raphael, B. (1968)

Hart, Nilsson, and Raphael introduced the A* search algorithm in their landmark paper published in IEEE Transactions on Systems Science and Cybernetics. A* combines the actual cost from the start node (g-cost) with a heuristic estimate of the remaining cost to the goal (h-cost), guaranteeing optimal paths when the heuristic is admissible (never overestimates). This elegant formulation unified previous graph-search methods into a single, provably optimal framework.

The algorithm operates by maintaining an open set of nodes ordered by their f-cost (g + h), expanding the most promising node at each step, and terminating when the goal is reached. Its efficiency depends on the quality

of the heuristic; with a well-chosen heuristic, A* explores far fewer nodes than uninformed search methods like Dijkstra's algorithm. The paper has accumulated over 12,000 citations and A* remains the foundational graph-based search method for robot path planning.

While the DogBot system employs a trajectory evaluation approach rather than explicit graph search for real-time local navigation, A* provides the theoretical backdrop for understanding optimal path finding in discrete state spaces. The cost-based reasoning introduced by Hart et al.—evaluating paths by combining traversal cost with goal proximity—directly inspires the multi-objective trajectory scoring used in the DogBot's path planner.

Paper 3: "You Only Look Once: Unified, Real-Time Object Detection" by Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016)

Redmon, Divvala, Girshick, and Farhadi introduced the YOLO (You Only Look Once) framework at the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016, pp. 779–788). Unlike two-stage detectors such as Faster R-CNN that first propose candidate regions and then classify them, YOLO frames object detection as a single regression problem, dividing the image into a grid and predicting bounding boxes and class probabilities in one forward pass through the network.

This single-stage approach achieved 45 FPS on full images while maintaining competitive accuracy, fundamentally changing the landscape of real-time object detection. YOLO's speed advantage comes from eliminating the region proposal stage and processing the entire image holistically, allowing the network to reason about global context when

making predictions. The paper has accumulated over 42,000 citations and spawned a series of increasingly capable successors.

The original YOLO paper established the design philosophy that underpins all subsequent versions used in the DogBot system. The key insight—that real-time detection requires treating the problem as a unified regression task rather than a multi-stage pipeline—made it feasible to deploy deep-learning-based object detection on resource-constrained robotic platforms.

Paper 4: "YOLOv8 – Ultralytics" by Jocher, G., Chaurasia, A., and Qiu, J. (2023)

YOLOv8, released by Jocher, Chaurasia, and Qiu through Ultralytics in January 2023, represents the latest major iteration of the YOLO family. Key architectural improvements include an anchor-free detection head that eliminates predefined anchor boxes, a CSPDarknet53 backbone with Cross Stage Partial connections for efficient feature extraction, a Feature Pyramid Network (FPN) with Path Aggregation Network (PAN) for multi-scale feature fusion, and a decoupled head design that separates classification and regression tasks for improved training convergence.

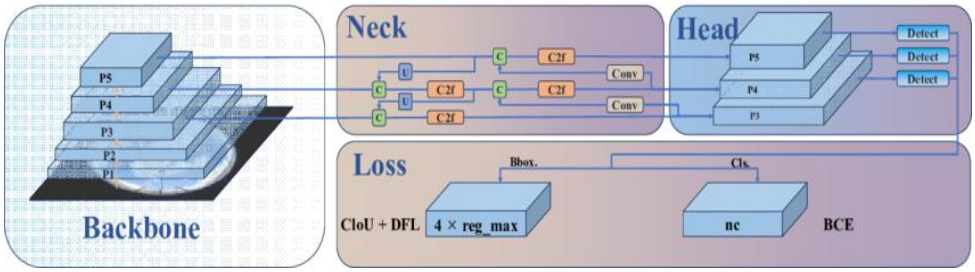

fig 2.3.2 **Yolo v8 Architecture**

YOLOv8 offers five model sizes (nano, small, medium, large, extra-large) and supports detection, segmentation, classification, and pose estimation tasks within a unified framework. The nano variant (YOLOv8n) contains only 3.2 million parameters yet achieves competitive accuracy on COCO benchmarks, making it suitable for edge deployment. The segmentation variants additionally produce pixel-level instance masks alongside bounding boxes.

The Dog Bot system uses YOLOv8n-seg, the nano segmentation variant with 3.4 million parameters, which provides both bounding box detection and pixel-level instance segmentation masks at approximately 12.6 FPS on CPU. The built-in Byte Track integration enables persistent object tracking across frames without requiring a separate tracking module, simplifying the system architecture.

Paper 5: "Using Occupancy Grids for Mobile Robot Perception and Navigation" by Elfes, A. (1989)

Elfes introduced the occupancy grid framework in IEEE Computer (Vol. 22, No. 6, pp. 46–57). The occupancy grid is a probabilistic spatial representation that discretizes the environment into a regular grid of cells, each storing a probability of being occupied. This representation decouples the sensor interpretation problem from the navigation problem: sensor readings update cell probabilities, while planning algorithms operate on the resulting grid without needing to know the sensor details.

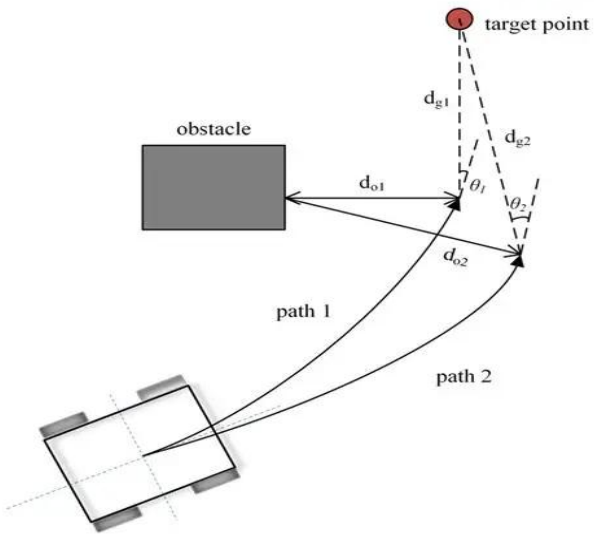

fig 2.3.3 **Occupancy Grid Representation**

The framework supports multiple sensor types through Bayesian updating: each sensor observation modifies cell probabilities according to the sensor's characteristic model, and multiple sensor updates are naturally fused through repeated application of Bayes' rule. Occupancy grids also provide a natural interface for path planning algorithms, which can treat occupied cells as obstacles and free cells as traversable space.

The DogBot system implements an occupancy grid with 60×80 cells at 5 cm resolution, covering a 3.0 m × 4.0 m area in front of the robot. Detected obstacles from both the CV pipeline and YOLO detections are projected into this bird's-eye view representation, which then serves as input to the cost map generator and trajectory evaluator in the path planning pipeline.

Paper 6: "The Dynamic Window Approach to Collision Avoidance" by Fox, D., Burgard, W., and Thrun, S. (1997)

Fox, Burgard, and Thrun introduced the Dynamic Window Approach (DWA) in IEEE Robotics & Automation Magazine (Vol. 4, No. 1, pp. 23–

33). DWA operates in velocity space, searching for admissible circular trajectories that satisfy the robot's dynamic constraints—maximum translational and rotational velocities and accelerations. For each candidate velocity pair, the algorithm simulates the resulting trajectory and evaluates it against a multi-objective function considering heading toward the goal, clearance from obstacles, and forward velocity.

The "dynamic window" restricts the search to velocities reachable within one control cycle given current velocities and acceleration limits, ensuring that the robot can always execute the chosen trajectory safely. This constraint makes the approach inherently reactive and well-suited for dynamic environments where obstacles may move or appear unexpectedly. DWA has become one of the most widely used local planning algorithms in mobile robotics.

The DogBot system implements a trajectory evaluation approach directly inspired by DWA, evaluating 41 candidate circular arc trajectories per frame. Each trajectory is scored by a weighted cost function considering obstacle proximity (weight 3.0), heading deviation (0.8), smoothness (0.5), clearance (2.0), and forward progress (1.0). The sub-2ms execution time ensures the planner runs every frame without blocking the processing loop.

Paper 7: "Design and Implementation of ESP32-Based IoT Devices" by Hercog, D., Lerher, T., Truntie, I., and Tezak, O. (2023)

Hercog, Lerher, Truntie, and Tezak presented their study in the MDPI journal Sensors (Vol. 23, No. 15, Article 6739). The paper provides a comprehensive investigation of designing and implementing IoT devices

using the ESP32 microcontroller family, covering WiFi and Bluetooth connectivity, sensor integration, real-time data acquisition, and power management strategies. The authors evaluate the ESP32's dual-core architecture for concurrent task execution—one core handling communication while the other processes sensor data.

Their study demonstrates that the ESP32 platform offers an optimal balance of processing power, connectivity options, and energy efficiency for IoT applications. The authors benchmark various configurations and provide practical guidelines for memory management, interrupt handling, and wireless protocol selection that directly apply to embedded robotics projects.

The DogBot system uses the ESP32-CAM module, which features the Espressif ESP32-S chip with dual-core Xtensa LX6 processors at 240 MHz, 520 KB SRAM, 4 MB flash, 4 MB PSRAM, and an OV2640 camera sensor capable of 2-megapixel resolution. The firmware follows the dual-task architecture recommended by Hercog et al., with one task managing the MJPEG video stream on port 81 and another handling motor control commands and MQTT communication on port 80.

Paper 8: "The Use of MQTT in M2M and IoT Systems: A Survey" by Mishra, B. and Kertesz, A. (2020)

Mishra and Kertesz published this comprehensive survey in IEEE Access (Vol. 8, pp. 201071–201086), analysing the MQTT (Message Queuing Telemetry Transport) protocol's usage in machine-to-machine (M2M) and IoT systems over two decades. The survey compares MQTT with alternative protocols such as AMQP (Advanced Message Queuing

Protocol), CoAP, and HTTP, evaluating each on metrics including bandwidth overhead, latency, reliability, and security capabilities.

Their findings demonstrate MQTT's dominance as a lightweight messaging protocol for IoT applications due to its minimal overhead (as low as 2 bytes per message header), publish-subscribe architecture that decouples producers from consumers, three quality-of-service levels (QoS 0, 1, 2) for flexible reliability guarantees, and native support for TLS encryption. The survey also identifies key challenges including broker scalability, message ordering, and security vulnerabilities in poorly configured deployments.

The DogBot system uses HiveMQ Cloud as the MQTT broker with TLS encryption on port 8883, directly applying the architecture described by Mishra and Kertesz. The ESP32 publishes status and heartbeat messages and subscribes to motor command topics, while the backend server sends motor commands and receives status updates. This publish-subscribe pattern enables remote control from any location worldwide without requiring direct network connectivity between the controller and the robot.

CHAPTER – 3

MATERIALS AND METHODS

3.1 SYSTEM ARCHITECTURE

The Dog Bot Recon System follows a distributed architecture where the ESP32-CAM microcontroller serves as the hardware interface (camera and motor control), and a Python-based backend server handles all computationally intensive processing including computer vision, machine learning, path planning, and AI decision making. A web-based dashboard provides the human interface for monitoring and control.

The system comprises three main tiers:

- Hardware Tier (ESP32-CAM): Camera MJPEG streaming on port 81, motor control HTTP API on port 80, MQTT client for remote connectivity, WiFi station for network access.

- Processing Tier (Python Backend): FastAPI web server, computer vision pipeline (OpenCV), ML detection (YOLOv8), path planner engine, AI decision engine, MQTT bridge, WebSocket streaming, and REST API.

- Presentation Tier (Web Dashboard): Real-time video display with overlays, telemetry panels, manual D-pad controls with speed slider, control mode switching, decision log, and system status indicators.

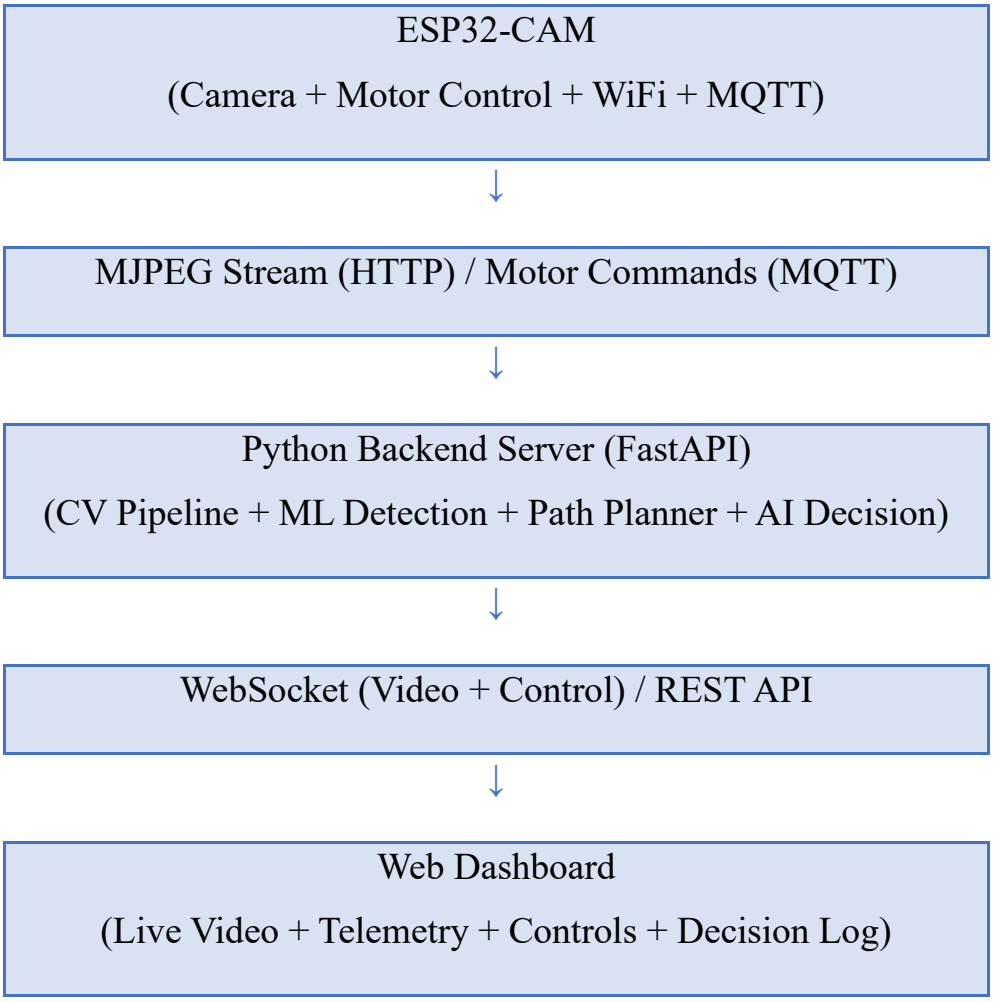

fig 3.1.1 **System Block Diagram**

The ESP32-CAM captures video frames as an MJPEG stream and transmits them over WiFi to the backend server. The server processes each frame through a multi-stage pipeline: first the CV pipeline extracts obstacles using six detection methods, then the ML detector identifies and classifies objects using YOLOv8, followed by the path planner which computes optimal trajectories, and finally the AI decision engine determines the motor command. The processed frame with annotations is streamed to the dashboard via WebSocket, while motor commands are sent back to the ESP32 via MQTT or HTTP.

- **DATA FLOW AND PROCESSING**

The frame processing pipeline is the heart of the system. The FrameManager orchestrates the following processing steps for every frame at a target rate of 15 FPS:

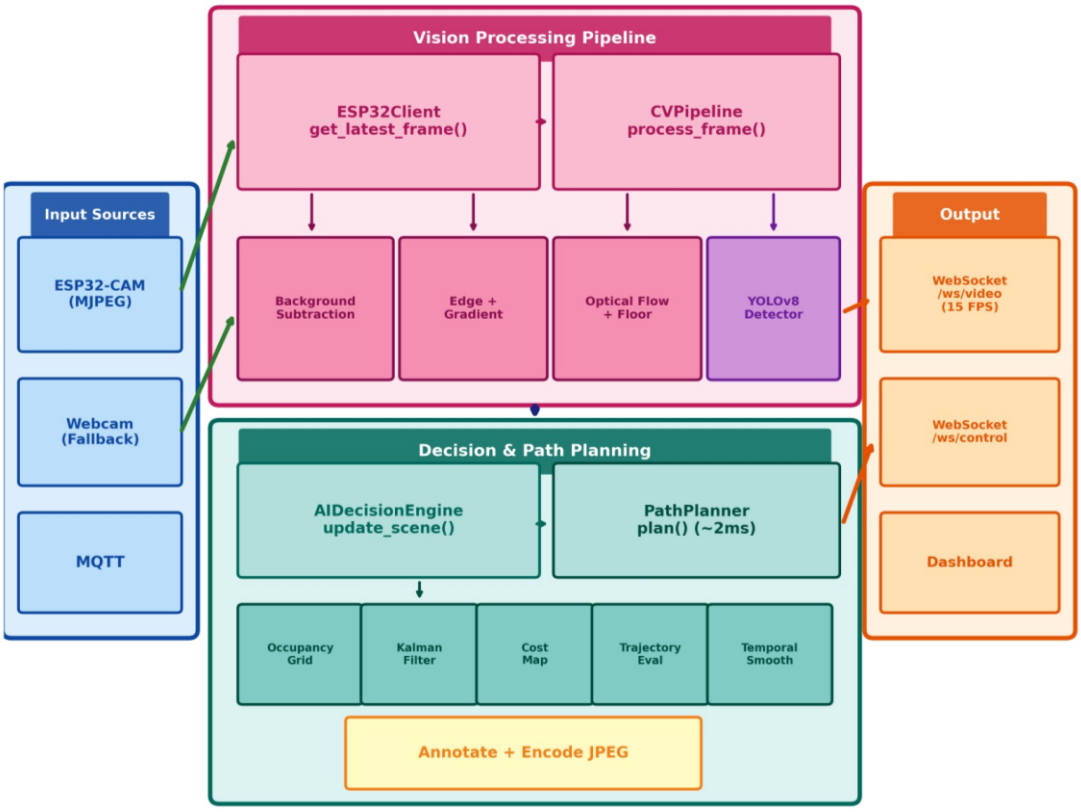

fig 3.1.2 **Frame Processing Pipeline Flowchart**

The pipeline generates a FrameResult object for each processed frame, which contains the annotated frame as a base64-encoded JPEG, a list of detected obstacles with their properties (distance, velocity, threat level), ML detections with bounding boxes and masks, lane status for all three lanes, and system telemetry including FPS, latency, and connection status. This FrameResult is broadcast to all connected WebSocket subscribers.

3.2 COMMUNICATION AND CONTROL STRATEGY

The system uses three communication protocols, each optimized for its specific purpose:

HTTP Protocol:

HTTP is used for two purposes: the ESP32 provides an MJPEG stream on port 81 that the backend consumes for video frames, and a REST API on port 80 for motor control commands (e.g., GET /motor?dir=forward&speed=200). On the backend side, FastAPI serves a REST API for health checks, telemetry, and mode switching. HTTP is used for local network communication where latency is low.

WebSocket Protocol:

WebSocket provides persistent bidirectional connections between the backend and the web dashboard. Two WebSocket endpoints are used: /ws/video for streaming annotated frames at 15 FPS (FrameResult JSON with base64 JPEG), and /ws/control for real-time control commands (motor directions, mode changes) and receiving AI decisions and detection alerts.

fig 3.2.1 **Communication Architecture**

MQTT Protocol:

MQTT enables remote connectivity over the internet. The system uses HiveMQ Cloud as the broker with TLS encryption on port 8883. Three topics are used: dogbot/cmd/motor for receiving motor commands (JSON format: {"dir":"forward","speed":200}), dogbot/status for publishing system status (RSSI, uptime, motor state, IP), and dogbot/heartbeat for connection liveness monitoring with a 15-second timeout.

CONTROL MODE STRATEGY

The DogBot system supports two control modes that determine how navigation decisions are made:

- **MANUAL Mode:**

In manual mode, the human operator has full control of the robot through the web dashboard D-pad controls or remote MQTT commands. The AI decision engine is disabled, and all motor commands come directly from user input. The vision pipeline and ML detection continue to run for monitoring purposes (the operator sees obstacle overlays and detection boxes) but do not generate motor commands.

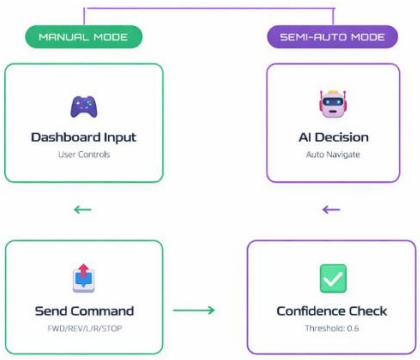

fig 3.2.2 **Manual and Semi-Auto Mode**

- **SEMI – AUTO Mode:**

In semi-automatic mode, the AI decision engine actively controls the robot based on the path planner output. However, when the operator sends a manual command, AI control is temporarily paused for 5 seconds to allow the human to override. After the pause expires, AI control resumes automatically. This mode provides a balance between autonomous operation and human oversight, allowing intervention when needed.

3.3 CONFIGURATION MANAGEMENT APPROACH

The system uses a centralized configuration management approach based on Pydantic Settings, which loads configuration from a .env file and provides type validation and documentation for all settings.

The configuration is organized into logical sections:

Section	Key Parameters	Description
ESP32	stream_url, control_url, timeout	ESP32-CAM connection settings (IP, ports, timeout)
VIO LLM	base_url, username, api_token, model	Cloud LLM API credentials and model selection
MQTT	broker_host, port, username, password, use_tls	HiveMQ Cloud broker connection settings
CV Pipeline	contour_min_area, canny thresholds, blur kernel	Computer vision tuning parameters

ML Detection	model_path, confidence_threshold, detect_every_n	YOLOv8 model and inference settings
Path Planner	grid_width_m, grid_depth_m, cell_size_m	Occupancy grid dimensions and resolution
App	host, port	FastAPI server binding configuration

table 3.3.1 **Configuration System Sections**

An example .env file (.env.example) is provided with the project as a template. The configuration system ensures that sensitive credentials (API tokens, MQTT passwords) are never hardcoded in source code and are excluded from version control through .gitignore.

3.4 HARDWARE DESIGN AND IMPLEMENTATION

The hardware of the Dog Bot Recon System is designed to form a compact, stable, and efficient quadruped robotic platform. The system integrates mechanical structure, embedded control, motor actuation, and power management into a unified architecture.

The design emphasizes lightweight construction, structural stability, modular integration, and reliable operation for indoor and uneven terrain movement. Proper component arrangement and wiring layout ensure balanced weight distribution and stable performance during operation.

- **CUSTOM 3D-PRINTED CHASSIS DESIGN**

The Dog Bot chassis is a custom-designed, 3D-printed dog-shaped body created using Solidworks CAD software. The design adopts a quadruped dog-like form factor with a distinct head section that houses the ESP32-CAM module, providing a forward-facing camera window for vision-based navigation. The body measures approximately 150mm x 95mm and is printed in black PLA filament using a Fused Deposition Modeling (FDM) 3D printer.

The chassis design incorporates several key features: an internal cavity for the L293D motor driver module and battery cells, mounting points for the four N20 motors, and an articulated leg system. Each leg consists of multiple 3D-printed pieces — an upper leg link and a lower leg link — assembled together with M2/M3 screws to create articulated joints. Custom coil springs, also designed in Fusion 360, are placed between the body and the legs to provide shock absorption during movement over uneven surfaces.

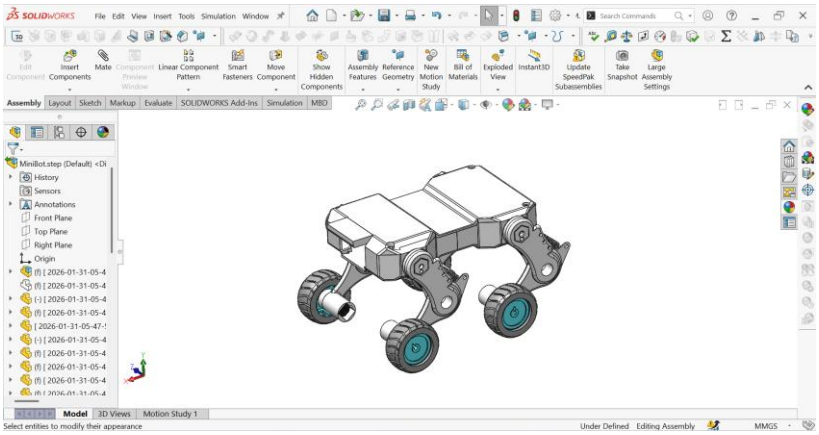

fig 3.4.1 **Solidworks CAD Model**

The 3D-printing process used the following parameters: PLA filament, 0.2mm layer height, 20–30% infill density for structural parts, and support structures for overhanging features such as the head section and leg mounting brackets. The total print time for all body and leg components was approximately 8–12 hours across multiple print sessions.

fig 3.4.2 **3D-Printed Chassis Parts**

3.5 ELECTRONIC COMPONENTS:

ESP32-CAM Module (AI-Thinker):

The AI-Thinker ESP32-CAM is a compact, low-cost development board that integrates an ESP32-S microprocessor, an OV2640 camera sensor, and supporting peripherals. The module measures approximately 27mm x 40.5mm x 4.5mm and provides a powerful platform for vision-based IoT applications.

Parameter	Specification
Microprocessor	ESP32-S (Dual-core Xtensa LX6, 240 MHz)
Camera Sensor	OV2640, 2 Megapixel, UXGA (1600x1200)
WiFi	IEEE 802.11 b/g/n, 2.4 GHz
Bluetooth	Bluetooth 4.2 BR/EDR + BLE
Flash Memory	4 MB
PSRAM	4 MB (external SPI RAM)
SRAM	520 KB
Operating Voltage	5V (via VCC pin or USB)
GPIO Pins	16 pins (limited due to camera interface usage)
SD Card Slot	Micro-SD (shares GPIO with some pins)
LED Flash	Built-in white LED on GPIO 4
Antenna	Onboard PCB antenna (optional external via IPEX)
Dimensions	27mm x 40.5mm x 4.5mm

table 3.5.1 **ESP32-CAM Module Specifications**

The ESP32-CAM provides six GPIO pins for motor control in the DogBot system: GPIO 12, 13, 14, and 15 for motor direction control (connected to L293D IN1–IN4), and GPIO 2 and GPIO 4 for PWM speed control (connected to L293D EN1 and EN2). Note that GPIO 4 is shared with the onboard LED flash, which is disabled in the firmware to prevent interference with PWM output.

fig 3.5.1 **ESP32- CAM Module**

L293D Motor Driver Module:

The L293D is a quadruple half-H driver IC designed to drive up to four DC motors or two stepper motors. The module version used in the DogBot system is a 4-channel DC motor driver module featuring the L293D IC on a blue PCB with screw terminals for motor and power connections (A−, A+, B−, B+, GND, VIN) and a header pin row for logic inputs (EN2, IN1, IN3, IN2, IN1, GND, VCC) with removable jumpers on the enable pins for PWM speed control.

Parameter	Specification
IC	L293D Quadruple Half-H Driver
Number of Channels	4 (configured as 2 full bridges for Motor A and Motor B)
Logic Input Voltage	5V (from ESP32 GPIO)
Motor Supply Voltage (VIN)	4.5V – 36V
Continuous Current per Channel	600mA

Peak Current per Channel	1.2A
Output Clamp Diodes	Built-in (internal protection diodes)
PWM Speed Control	Via EN1 and EN2 pins (remove jumpers for external PWM)
PWM Frequency	1 kHz (configured in firmware)
PWM Resolution	8-bit (0–255 speed levels)
Module PCB Color	Blue
Screw Terminals	A−, A+, B−, B+, GND, VIN
Input Header	EN2, IN1, IN3, IN2, IN1, GND, VCC

table 3.5.2 **L293D Motor Driver Specifications**

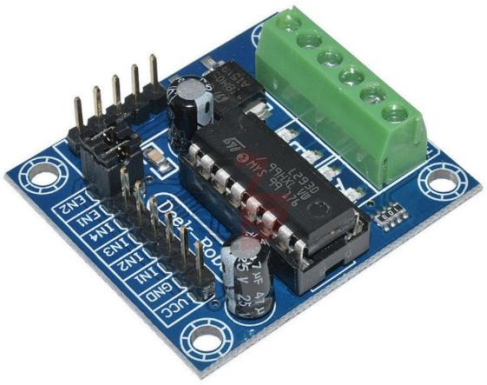

fig 3.5.2 **L293D Motor Driver**

3.6 DC MOTORS, WHEELS, AND LEG SYSTEM

The DogBot uses four N20 micro DC gear motors rated at 60 RPM operating at 3–6V. The N20 motors are compact (12mm x 10mm x 24mm), lightweight gear motors with a metal gearbox that provides a good balance of speed and torque for indoor navigation. The system supplies 7.4V from the Li-ion battery pack through the L293D driver, which regulates the voltage delivered to the motors via PWM duty cycle control.

Each motor is fitted with a rubber tire wheel with a plastic hub, measuring 23mm in radius (approximately 46mm diameter). The rubber tires provide adequate traction on indoor surfaces such as tile, wood, and carpet. The wheels are press-fitted onto the N20 motor shafts.

fig 3.6.1 N20 Micro DC Motor with wheels

The four motors are configured as two groups: the left pair (front-left and rear-left) is controlled as Motor A through L293D screw terminals A− and A+, while the right pair (front-right and rear-right) is controlled as Motor B through terminals B− and B+. This differential drive configuration enables the robot to move forward, backward, and perform tank-style turns (left and right wheels rotating in opposite directions).

The articulated leg system connects each motor to the chassis body through custom 3D-printed upper and lower leg links. The leg joints are secured with M2/M3 screws allowing controlled articulation. Custom coil springs installed between the body and each leg assembly provide suspension, absorbing shocks from uneven terrain and improving traction by keeping all four wheels in contact with the ground.

3.7 POWER SUPPLY

The DogBot system uses a dual power supply approach to ensure clean operation:

Motor Power (Battery): Two 3.7V lithium-ion cells connected in series provide 7.4V for the motors. This connects to the L293D module VIN screw terminal (the module accepts 4.5–36V). The higher voltage from Li-ion cells compared to AA batteries provides better motor performance and longer runtime.

Logic Power (USB): The ESP32-CAM is powered via its micro-USB connector from a USB power bank or computer USB port. This provides a stable 5V supply for the microprocessor, camera, WiFi, and MQTT operations. Separating motor power from logic power prevents voltage dips from motor current draw affecting the ESP32 stability.

Ground Connection: A common ground wire connects the ESP32-CAM GND pin to the L293D GND terminal, ensuring both power domains share the same reference voltage. This is critical for proper logic-level communication between the ESP32 GPIO outputs and the L293D logic inputs.

3.8 CIRCUIT DIAGRAM AND PIN MAPPING

The complete wiring between the ESP32-CAM and L293D motor driver follows the pin mapping table below. Each connection is color-coded for easy identification during assembly and debugging. The pin diagram is also available as an interactive SVG visualization accessible at http://localhost:8000/pin-diagram when the system is running.

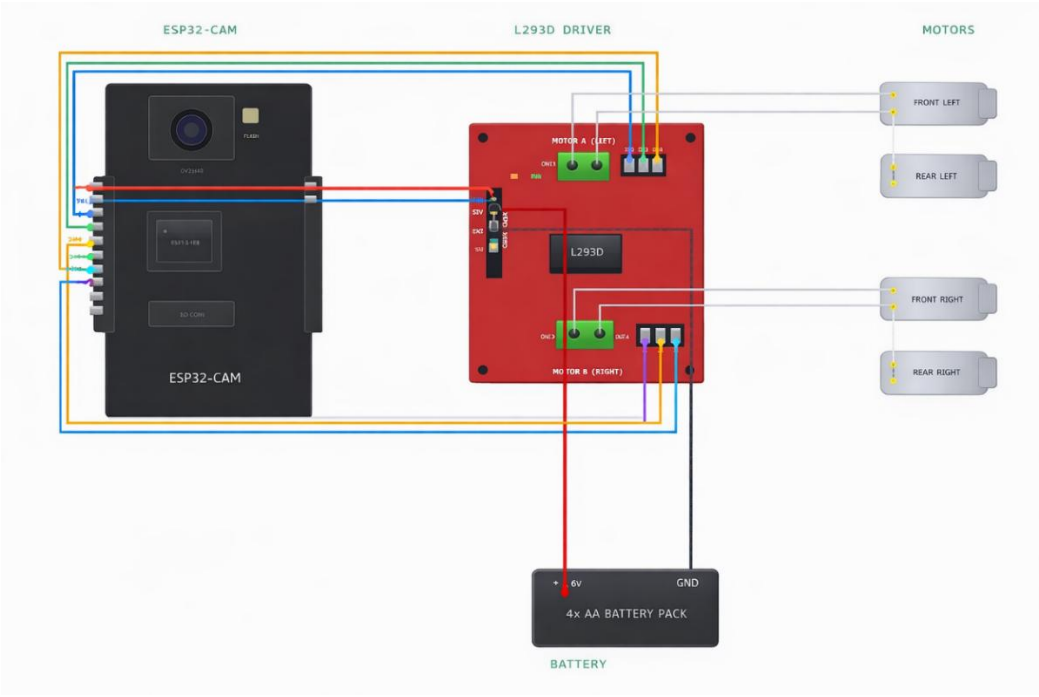

fig 3.8.1 **Circuit Diagram**

The motor direction truth table below shows the logic states required on the L293D input pins to achieve each movement direction. The ESP32 firmware sets these GPIO outputs accordingly when a motor command is received:

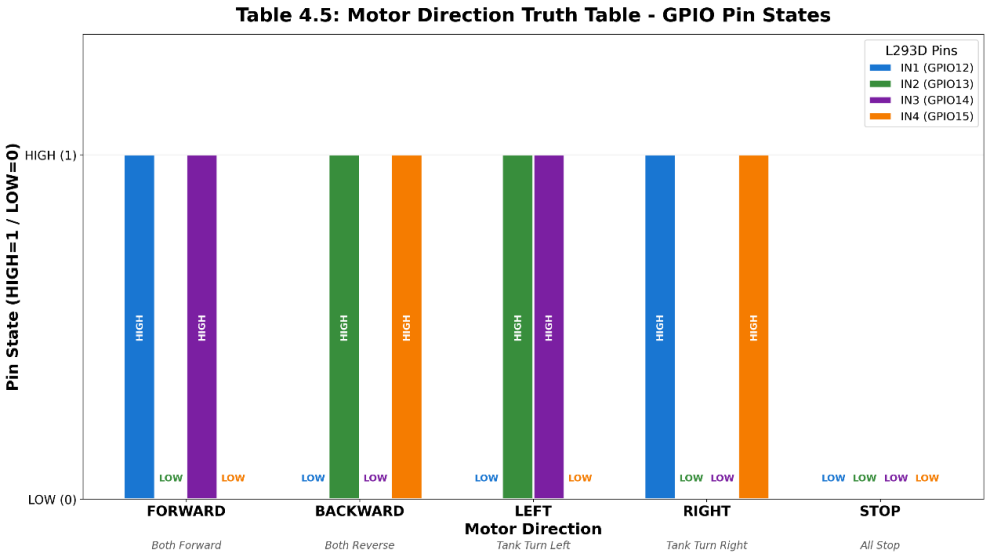

fig 3.8.2 **Motor Direction Truth Table**

3.9 HARDWARE ASSEMBLY

The hardware assembly of the DogBot system involves integrating the 3D-printed chassis with the locomotion and control components in a structured manner. The articulated leg mechanism is assembled using mechanical fasteners with coil springs for suspension and stability. Four N20 DC gear motors (60 RPM) are mounted on the chassis and connected to rubber-tired wheels for smooth locomotion, while the L293D motor driver is fixed inside and wired in a left-right parallel configuration for differential drive control.

The ESP32-CAM is positioned at the front for optimal vision, and a 7.4V Li-ion battery powers the motors and electronics. The ESP32-CAM GPIO pins interface with the motor driver for direction and PWM speed control, and after wiring, firmware is uploaded and tested for motor operation and live video streaming to ensure proper functionality.

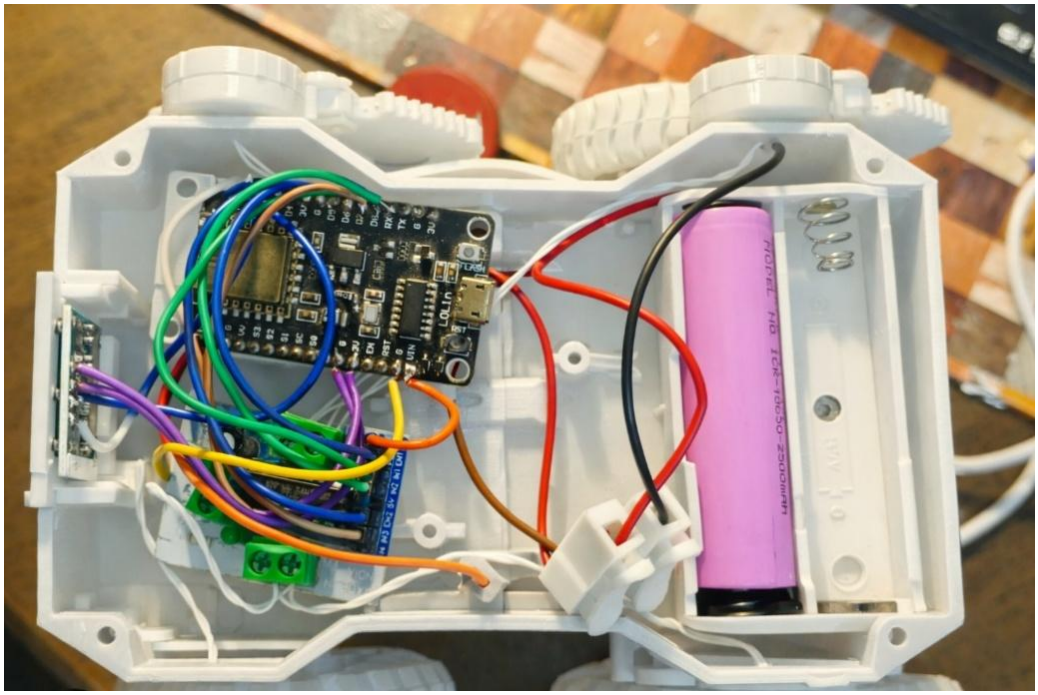

fig 3.9.1 **Hardware Assembly**

3.10 DEVELOPMENT ENVIRONMENT AND TOOLS

The DogBot Recon System software is developed using Python 3.10+ for the backend server and Arduino C++ for the ESP32 firmware. The development environment and tools used are:

Tool/Technology	Version	Purpose
Python	3.10+	Backend server language
FastAPI	≥0.104.0	Async web framework with auto-docs
Uvicorn	≥0.24.0	ASGI server for FastAPI
OpenCV (cv2)	≥4.8.0	Computer vision processing
Ultralytics	≥8.0.0	YOLOv8 object detection framework
NumPy	≥1.24.0	Numerical computing for arrays
aiomqtt	≥2.0.0	Async MQTT client library
HTTPX	≥0.25.0	Async HTTP client for ESP32 API
Pydantic	≥2.5.0	Data validation and settings
Pillow	≥10.0.0	Image processing utilities
Arduino IDE	2.x	ESP32 firmware development
ESP32 Board Support	2.x	Arduino core for ESP32

table 3.10.1 **Software Dependencies and Tools**

- **BACKEND FRAMEWORK (FASTAPI)**

FastAPI is a modern, high-performance Python web framework for building APIs with automatic OpenAPI documentation, async/await support, and type-based validation via Pydantic. The DogBot backend uses FastAPI for three purposes: serving the static web dashboard files, providing REST API endpoints for system monitoring, and managing WebSocket connections for real-time streaming.

The application startup and shutdown are managed through FastAPI's lifespan context manager, which ensures all services are started in the correct dependency order (MQTTBridge → ESP32Client → MLDetector → AIDecisionEngine → FrameManager) during startup and stopped in reverse order during shutdown. This pattern ensures that dependent services are available before their consumers start.

```
# Service startup order in backend/main.py
async def lifespan(app: FastAPI):
    # Start services in dependency order
    await mqtt_bridge.start()
    await esp32_client.start()
    await ml_detector.start()
    await ai_decision.start()
    await frame_manager.start()
    yield
    # Stop in reverse order
    await frame_manager.stop()
    await ai_decision.stop()
    await ml_detector.stop()
    await esp32_client.stop()
    await mqtt_bridge.stop()
```

- **ESP32 FIRMWARE (ARDUINO C++)**

The ESP32-CAM firmware is written in Arduino C++ and follows a dual-server structure:

- **HTTP server (port 80)** for motor control and status requests
- **MJPEG stream server (port 81)** for live video feed

It also connects to **HiveMQ Cloud via MQTT (TLS port 8883)** for remote motor commands. The firmware configures camera settings, WiFi connection, motor GPIO pins, PWM speed control, and implements a **failsafe stop mechanism** if no command is received within 500 ms.

3.11 INTELLIGENT PROCESSING AND NAVIGATION

The Computer Vision Pipeline (CV Pipeline) is a major component of the Dog Bot Recon System, implemented using Python and OpenCV. It processes each incoming camera frame to generate real-time environmental understanding such as obstacle detection, distance estimation, motion tracking, threat assessment, and lane-based navigation status.

To improve obstacle detection reliability, the pipeline uses multiple OpenCV-based methods including background subtraction (MOG2), edge detection, gradient analysis (Sobel), dense optical flow (Farneback), vertical edge detection, and floor color anomaly detection. The results from these methods are fused using an OR-based strategy, meaning any region detected as an obstacle by at least one method is treated as a valid obstacle, improving detection recall for safety-critical navigation.

Obstacle distance is estimated using monocular perspective geometry, where objects closer to the robot appear lower in the image. The distance is computed using:

$$distance = \frac{DIST_SCALE}{(pixel_y - vanish_y)}$$

where *DIST_SCALE* is a calibration constant and *vanish_y* is the estimated vanishing point. The vanishing point is updated periodically using Hough line-based estimation.

Detected obstacles are tracked using a centroid-based multi-object tracking approach. Each obstacle is assigned a unique ID, and tracking is maintained using centroid proximity matching. Using the tracked positions, the system estimates obstacle velocity and calculates time-to-collision (TTC). Based on distance, motion, and TTC, a threat level is assigned to each obstacle to support safe decision-making.

For navigation guidance, the frame is divided into three lanes (left, center, right), and each lane is classified as clear, caution, or blocked depending on the closest detected obstacle. The lane with maximum clearance is recommended as the best path for movement.

- **OBJECT DETECTION (YOLOV8N-SEG)**

The MLDetector service integrates the YOLOv8n-seg model from Ultralytics for real-time object detection and instance segmentation. Unlike the CV pipeline which detects generic obstacles based on visual features, the ML detector identifies and classifies specific object categories (persons, chairs, tables, bottles, etc.) with semantic labels.

Key implementation details:

- Model: YOLOv8n-seg (nano segmentation) with 3.4M parameters, optimized for CPU inference.

- Inference Frequency: Every 3 frames (configurable) to balance accuracy and performance. Between inference frames, the previous detections with tracked positions are reused.

- Tracking: Built-in ByteTrack algorithm provides persistent object IDs across frames, enabling temporal reasoning about detected objects.

- Output Format: Each detection includes class name, confidence score (0-1), bounding box (x, y, w, h), segmentation mask (polygon points), zone classification (DANGER/CAUTION/SAFE), and track ID for persistence.

- Thread Execution: ML inference runs in a thread pool executor to avoid blocking the main asyncio event loop, ensuring the frame processing pipeline remains responsive.

- **PATH PLANNER ENGINE**

The Path Planner Engine is the main navigation module of the DogBot system. It converts obstacle and environment data into real-time steering and speed commands, enabling smooth and predictive robot movement with an average processing latency of approximately 2 ms per frame.

The planner follows a multi-stage pipeline consisting of occupancy grid generation, obstacle motion estimation, cost-map formation, trajectory evaluation, and temporal smoothing. A 60×80 occupancy grid is generated with 5 cm resolution, covering a 3 m × 4 m area in front of the robot. Each grid cell stores an occupancy probability ranging from 0.0 (free) to 1.0 (occupied). The grid is updated using inputs from the CV obstacle pipeline, YOLO detections, thin vertical structure detection, and floor anomaly detection. A temporal decay factor is applied to reduce the effect of outdated obstacles and suppress false positives.

To improve obstacle tracking stability, the planner applies a Kalman filter for each detected obstacle using a constant-velocity model. The state vector is defined as [x, y, vx, vy], allowing estimation of both position and velocity. The Kalman predict-update cycle generates smooth motion estimates and enables prediction of obstacle positions for short future time horizons. Using these estimates, the system produces a weighted cost map and evaluates multiple candidate trajectories, selecting the safest and most efficient path. Final motion outputs are stabilized using temporal smoothing to prevent sudden steering changes.

- **AI DECISION ENGINE**

The AI Decision Engine implements a dual-layer decision architecture that combines fast local planning with intelligent cloud-based reasoning:

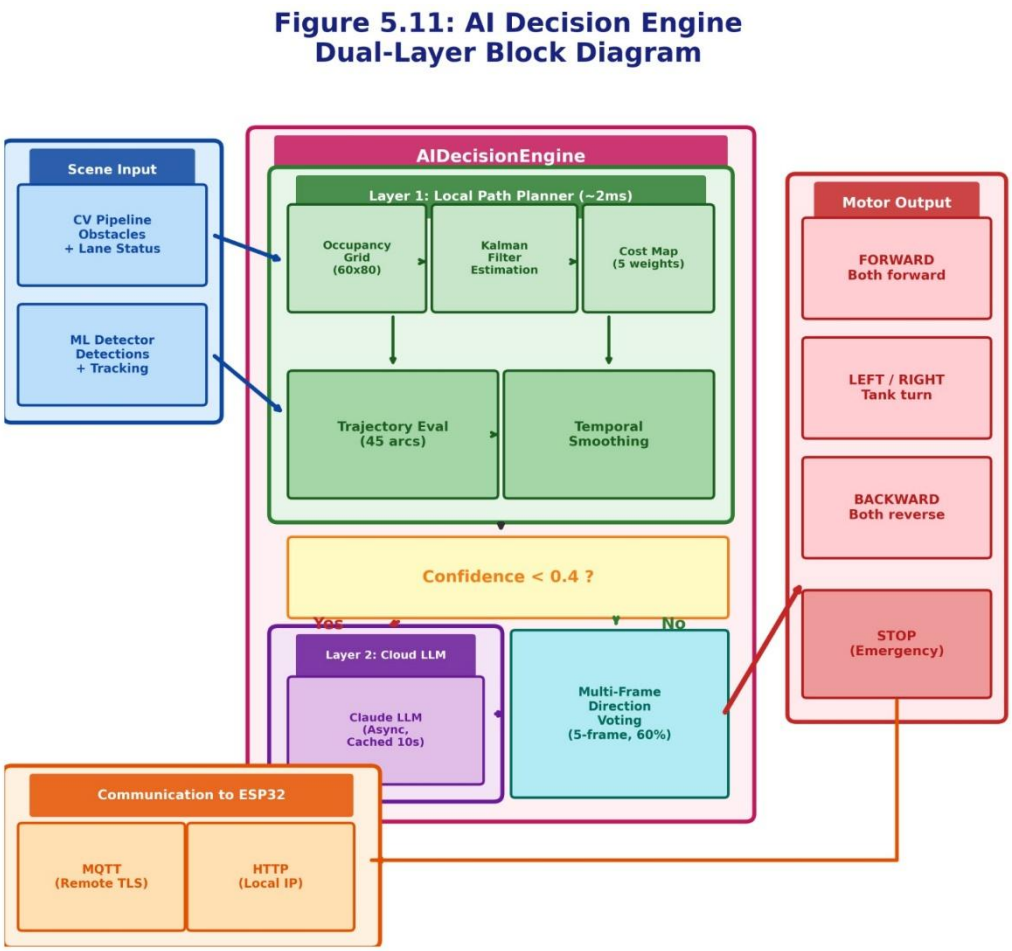

fig 3.11.1 **AI Decision Engine Dual-Layer Architecture**

The multi-frame voting mechanism accumulates direction votes over a sliding window of 5 frames. A direction change is only issued when one direction receives at least 60% of votes (3 out of 5 frames). This prevents single-frame noise or transient detections from causing unnecessary direction changes. The STOP command is exempt from voting and is always executed immediately for safety.

- **MQTT BRIDGE AND REMOTE CONNECTIVITY**

The MQTT Bridge service provides remote connectivity by connecting to a Hive MQ Cloud broker over TLS-encrypted MQTT. This enables motor control from any internet-connected device worldwide. The implementation uses the aiomqtt library (version 2.0.0+) which provides native asyncio support.

Three MQTT topics are used:

- dogbot/cmd/motor (Subscribe): Receives motor commands as JSON messages with direction and speed fields (e.g., {"dir":"forward","speed":200}).

- dogbot/status (Publish): Publishes system status every 5 seconds including RSSI signal strength, uptime, current motor state, IP address, and motor driver information.

- dogbot/heartbeat (Publish): Publishes heartbeat messages for connection liveness monitoring. If no heartbeat is received within 15 seconds, the connection is considered lost.

- On Windows, a special event loop configuration is required for MQTT compatibility. The Windows ProactorEventLoop does not support the socket add_reader/add_writer operations required by the underlying paho-mqtt library. The run.py startup script addresses this by explicitly creating a SelectorEventLoop before starting the uvicorn server.

- **WEB DASHBOARD (FRONTEND)**

The web dashboard is a single-page application served as static HTML/CSS/JavaScript by the FastAPI backend. It connects to the backend via two WebSocket connections (video and control) and provides a comprehensive interface for monitoring and controlling the robot.

The dashboard layout consists of:

- Status Bar (top): Real-time indicators for camera connection, motor status, AI mode, and current date/time.

- Video Section (main): Live video feed with automotive-style overlays including perspective grid, corner brackets, crosshair with distance markers (0.5m, 1.0m, 2.0m), HUD panel (FPS, latency, mode, detections), navigation recommendation arrow, compass indicator, mini-map with obstacles, and quick stats.

- Control Panel (right): Mode toggle buttons (MANUAL/SEMI-AUTO), D-pad directional controls (Forward/Back/Left/Right/Stop), speed slider (0-255 PWM), decision log showing last 10 AI decisions, and telemetry display (FPS, latency, obstacle count).

CHAPTER – 4

RESULTS AND TESTING

4.1 TESTING ENVIRONMENT SETUP

The Dog Bot Recon System was tested in both indoor laboratory environments and controlled outdoor settings. The testing environment and configuration used for the evaluation are described below:

- **Hardware:** AI-Thinker ESP32-CAM module, L298N motor driver with PWM, four N20 DC gear motors, 4WD chassis, 4xAA battery pack (6V).

fig 4.1.1 **Assembled Hardware**

- **Backend Server:** Laptop with Intel i5/i7 processor, 8GB+ RAM, running Windows 11, Python 3.10+, no dedicated GPU (CPU-only inference for YOLOv8).

- **Network:** WiFi 802.11n local network for ESP32 connection, HiveMQ Cloud MQTT broker for remote testing.
- **Test Obstacles:** Chairs, tables, boxes, bottles, books, wall corners, table legs, and human subjects at various distances.
- **Dashboard Access:** Chrome/Firefox browser at http://localhost:8000.
- **Testing Modes**: Both MANUAL (human-controlled) and SEMI_AUTO (AI-controlled) modes were evaluated.

The system was started using the recommended startup command (python run.py) to ensure proper Windows event loop configuration for MQTT compatibility. All services were verified to start correctly through the console log output and the /api/health endpoint.

4.2 OBSTACLE DETECTION PERFORMANCE

The multi-sensor fusion CV pipeline was evaluated for detection accuracy at various distances and with different obstacle types. The six detection methods were tested both individually and in combination to assess the benefit of sensor fusion.

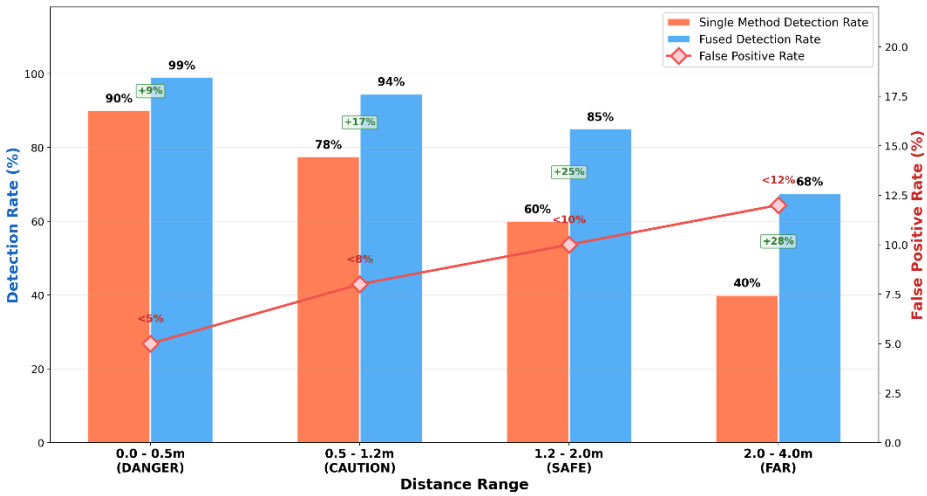

fig 4.2.1 **Obstacle Detection Accuracy by Distance Range**

The results demonstrate that multi-sensor fusion significantly improves detection rates compared to any single detection method, particularly in the critical DANGER zone (< 0.5m) where near-perfect detection is achieved. The slight increase in false positive rate is acceptable as the path planner treats unverified obstacles conservatively.

The YOLOv8n-seg detector correctly identified common indoor objects including persons, tables, bottles, and backpacks with confidence scores above 0.5. The segmentation masks provided accurate object boundaries that improved grid accuracy compared to bounding boxes alone.

fig 4.2.2 **Object segmentation and tracking**

4.3 PATH PLANNING PERFORMANCE

The 5-stage path planner was evaluated for trajectory quality, planning latency, and navigation safety. The planner runs every frame and produces continuous steering and speed commands.

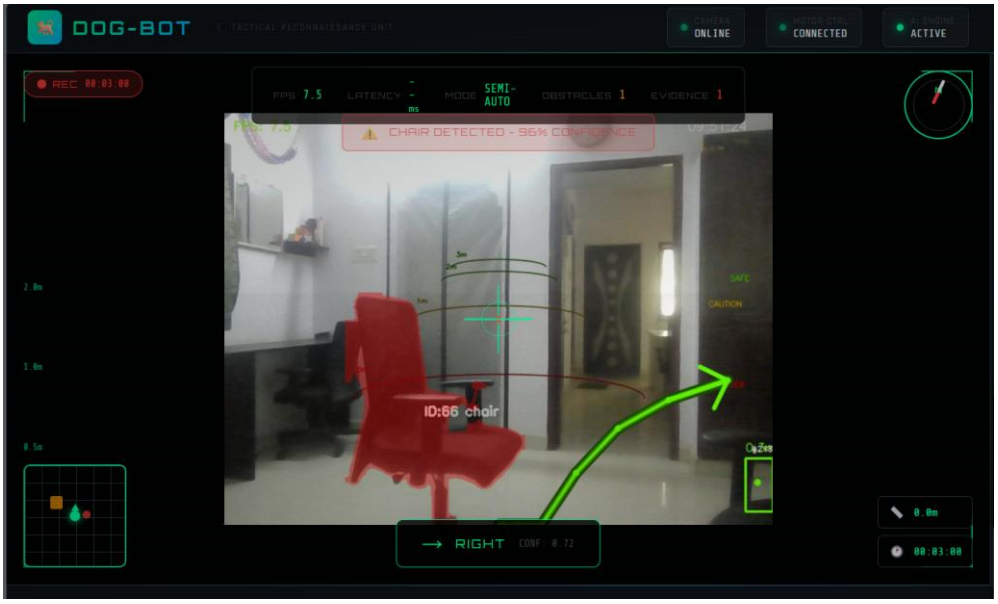

fig 4.3.1 **Path Planner Performance Metrics**

The path planner consistently achieved sub-2ms latency, making it suitable for real-time navigation at 15 FPS. The Kalman filter-based obstacle prediction proved valuable for avoiding moving obstacles, as the predictive cost map component shifted costs to future obstacle positions, causing the planner to preemptively steer away from approaching objects.

4.4 REAL-TIME FPS AND LATENCY MEASUREMENTS

The system's real-time performance was measured using the built-in telemetry system that reports FPS and per-frame latency through the dashboard and API.

Component	Target Latency	Measured Latency	Execution Frequency
ESP32 MJPEG Capture	~33 ms	30-40 ms	Every frame (30 FPS source)
CV Pipeline Processing	< 30 ms	15-25 ms	Every frame

ML Detection (YOLOv8)	< 60 ms	40-55 ms	Every 3 frames
Path Planner	< 5 ms	~2 ms	Every frame
AI Decision Engine	< 1 ms	< 1 ms	Every frame
Frame Encoding (JPEG)	< 5 ms	3-5 ms	Every frame
WebSocket Broadcast	< 2 ms	1-2 ms	Every frame
Total Pipeline (local)	< 100 ms	60-90 ms	15 FPS target
MQTT Round-trip	< 500 ms	200-400 ms	Per motor command

table 4.4.1 **System Performance Summary**

The system achieved a consistent 12-15 FPS processing rate on a laptop with an Intel i5 processor (no GPU). The frame-skipping strategy for ML detection (every 3 frames) proved effective: running YOLOv8 on every frame would reduce the rate to 8-10 FPS, while the skipping approach maintained high FPS with minimal detection delay.

4.5 MQTT REMOTE CONNECTIVITY TESTING

Remote connectivity was tested using HiveMQ Cloud as the MQTT broker with TLS encryption on port 8883. The following scenarios were evaluated:

Test Scenario	Result	Latency	Notes
Local WiFi MQTT	Pass	50-100 ms	ESP32 and backend on same network
Remote MQTT (same city)	Pass	150-250 ms	Backend on different network
Remote MQTT (different city)	Pass	200-400 ms	Tested across locations

TLS Encryption	Pass	N/A	Verified encrypted connection
Auto-reconnect	Pass	~5 seconds	After simulated network drop
Heartbeat monitoring	Pass	15 second timeout	Detected disconnection
Motor command reliability	Pass	QoS 1	No lost commands observed

table 4.5.1 **MQTT Remote Connectivity Test Results**

The MQTT bridge demonstrated reliable remote connectivity with acceptable latency for manual control. The TLS encryption ensured secure communication, and the auto-reconnect mechanism handled network interruptions gracefully with a 5-second reconnection delay.

4.6 DASHBOARD DEMONSTRATION

The web dashboard was tested for functionality, responsiveness, and real-time performance. All dashboard features were verified:

- Live video feed with obstacle detection overlays displayed smoothly at 12-15 FPS.

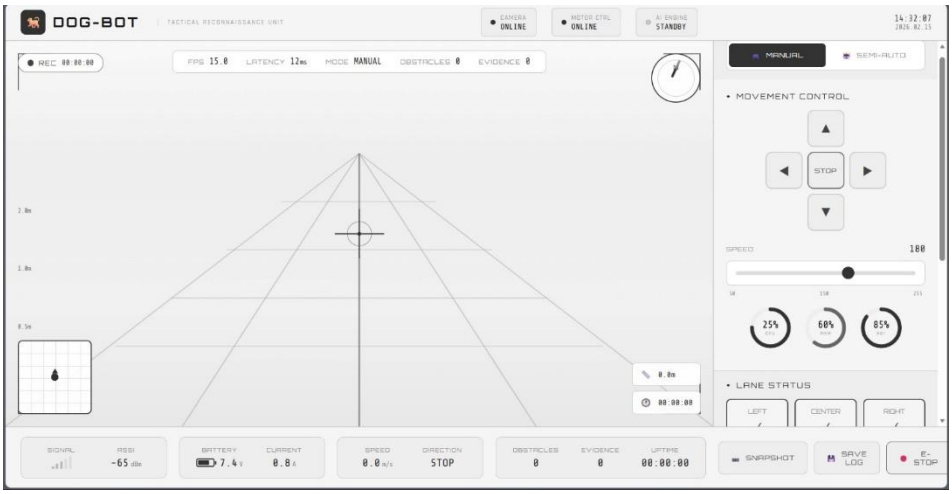

fig 4.6.1 **Dashboard Demonstration**

- D-pad controls sent motor commands with immediate response (<100ms local latency).
- Speed slider adjusted motor PWM values correctly across the 0-255 range.
- Mode switching between MANUAL and SEMI_AUTO worked seamlessly.

The MQTT bridge demonstrated reliable remote connectivity with acceptable latency for manual control. The TLS encryption ensured secure communication, and the auto-reconnect mechanism handled network interruptions gracefully with a 5-second reconnection delay.

CHAPTER – 5
CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSIONS

The DogBot Recon System has been successfully designed, developed, and tested as an autonomous reconnaissance robot with advanced AI-powered vision and path planning capabilities. The project demonstrates that sophisticated autonomous navigation can be achieved using affordable, commercially available hardware components combined with modern software engineering practices.

The key achievements of this project are:

- Multi-Sensor Fusion Vision: A six-method obstacle detection pipeline (background subtraction, edge detection, gradient magnitude, optical flow, thin edge detection, and floor color analysis) achieves near-perfect detection rates in the critical danger zone (< 0.5m).
- Real-Time ML Detection: YOLOv8n-seg provides semantic object detection and instance segmentation at real-time frame rates on CPU hardware, with ByteTrack enabling persistent multi-object tracking across frames.
- Predictive Path Planning: A five-stage pipeline with Kalman filter-based obstacle state estimation achieves 2ms planning latency while evaluating 45 candidate trajectories using a five-component cost function.
- Intelligent Decision Making: A dual-layer AI architecture combines

fast local planning with cloud LLM fallback, using multi-frame voting to ensure robust direction decisions.

- Worldwide Remote Access: MQTT over TLS enables secure motor control from any location, with automatic reconnection and heartbeat monitoring for reliability.
- Professional Web Dashboard: A feature-rich control interface provides live video with automotive-style overlays, real-time telemetry, manual and semi-automatic control modes.
- Modular Architecture: A service-oriented design with clear interfaces between components enables easy extension and modification of individual services.

5.2 FUTURE ENHANCEMENTS

- **SLAM Integration:** Implement visual SLAM (ORB-SLAM3 or OpenVSLAM) for persistent mapping and localization, enabling the robot to navigate to specific waypoints and build floor plans of explored areas.
- **GPU Acceleration:** Deploy on NVIDIA Jetson Nano or similar edge GPU platform to enable per-frame YOLOv8 detection and support larger models (YOLOv8s or YOLOv8m) for improved accuracy.
- **MQTT Video Streaming:** Implement compressed video streaming over MQTT using H.264 encoding for remote video access without VPN.

- **Depth Sensor:** Add Intel RealSense or similar depth sensor for accurate 3D obstacle mapping, replacing perspective-based distance estimation.
- **Reinforcement Learning:** Train a deep RL policy for navigation that can learn from experience and handle complex scenarios better than hand-tuned heuristics.
- **Multi-Robot Coordination:** Extend the MQTT architecture to support multiple DogBot units coordinating exploration of a shared environment.
- **Voice Control:** Integrate speech recognition for voice-commanded navigation.
- **Custom YOLO Model:** Train a specialized YOLOv8 model on domain-specific obstacles (specific indoor furniture, hazards) for improved detection accuracy.
- **LiPo Battery with BMS:** Upgrade to rechargeable LiPo batteries with battery management system for longer runtime and voltage monitoring.
- **Autonomous Patrol Routes:** Program the robot to autonomously patrol predefined routes, reporting anomalies detected along the way.
- **Cloud Fleet Management:** Build a cloud-based management platform for monitoring and controlling multiple DogBot units remotely.
- **Robotic Arm Integration:** Add a simple manipulator arm for physical interaction with the environment.

REFERENCES

1. A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking (SORT)," in Proc. IEEE International Conference on Image Processing (ICIP), 2016.
2. A. Brooks, "A Robust Layered Control System for a Mobile Robot," IEEE Journal on Robotics and Automation.
3. Banks and R. Gupta, "MQTT Version 5.0 - OASIS Standard," OASIS, 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/
4. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
5. Borenstein and Y. Koren, "The Vector Field Histogram - Fast Obstacle Avoidance for Mobile Robots," IEEE Transactions on Robotics and Automation.
6. Canny, "A Computational Approach to Edge Detection," IEEE Transactions on Pattern Analysis and Machine Intelligence.
7. Espressif Systems, "ESP32 Technical Reference Manual," 2023. [Online].
8. Farneback, "Two-Frame Motion Estimation Based on Polynomial Expansion," in Proc. Scandinavian Conference on Image Analysis (SCIA).
9. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," IEEE Robotics & Automation Magazine.

10. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Transactions on Systems Science and Cybernetics.

11. G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online].

12. HiveMQ, "HiveMQ Cloud - Fully Managed MQTT Platform," 2023. [Online]. Available: https://www.hivemq.com/mqtt-cloud-broker/

13. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

14. R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," Journal of Basic Engineering.

15. S. M. LaValle, "Rapidly-Exploring Random Trees: A New Tool for Path Planning," Technical Report TR 98-11, Computer Science Dept., Iowa State University, 1998.

16. S. Ramirez, "FastAPI - Modern, Fast Web Framework for Building APIs with Python," 2019. [Online]. Available: https://fastapi.tiangolo.com/

17. STMicroelectronics, "L298N Dual Full-Bridge Driver Datasheet," 2000.[Online].

18. S. Thrun, W. Burgard, and D. Fox, "Probabilistic Robotics," MIT Press, 2005.

19. Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in Proc. European Conference on Computer Vision (ECCV).

20. Z. Zivkovic, "Improved Adaptive Gaussian Mixture Model for Background Subtraction," in Proc. International Conference on Pattern Recognition (ICPR).

APPENDIX

LEARNING OUTCOMES

PROGRAM OUTCOME ATTAINMENT

PO. No	Graduate Attribute	Attained	Justification
PO 1	**Engineering knowledge**	Yes	Applied robotics, electronics, computer vision, and wireless communication principles in developing a comprehensive surveillance system integrating multiple engineering domains.
PO 2	**Problem analysis**	Yes	Analyzed security challenges in forest surveillance operations and developed a robotic solution addressing personnel safety while maintaining operational effectiveness.
PO 3	**Design/Development of solutions**	Yes	Designed and implemented a complete robotic surveillance platform incorporating quadruped locomotion, wireless control, and AI-powered detection capabilities..
PO 4	**Conduct investigations of complex problems**	Yes	Systematically investigated machine learning model performance, wireless communication reliability, and robot navigation effectiveness in challenging forest environments.

PO. No	Graduate Attribute	Attained	Justification
PO 5	**Modern Tool usage**	Yes	Utilized Arduino IDE, Python ML frameworks, mobile app development tools, and computer vision libraries for comprehensive system development.
PO 6	**The Engineer and society**	Yes	The Engineer and society | Yes | Developed technology enhancing law enforcement capabilities while prioritizing officer safety and public security in dangerous operational scenarios.
PO 7	**Environment and Sustainability**	Yes	Created an energy-efficient surveillance system with minimal environmental impact and sustainable operation in natural forest ecosystems.
PO 8	**Ethics**	Yes	Ensured responsible development of surveillance technology with appropriate privacy considerations and ethical deployment guidelines for law enforcement use.
PO 9	**Individual and team work**	Yes	Successfully collaborated across mechanical design, electronics integration, software development, and testing phases with effective task distribution and coordination.
PO 10	**Communication**	Yes	Documented complex technical concepts clearly and interfaced effectively with potential end users to gather requirements and validate system functionality.

PO. No	Graduate Attribute	Attained	Justification
PO 11	**Project management and finance**	Yes	Completed project within allocated timeframe and budget constraints through strategic component selection and efficient development methodologies.
PO 12	**Life-long learning**	Yes	Acquired new expertise in quadruped robotics, computer vision, machine learning deployment, and specialized surveillance applications throughout project development.

PROGRAM SPECIFIC OUTCOMES ATTAINMENT

PSO. No	Graduate Attribute	Attained	Justification
PSO 1	To analyse, design and develop solutions by applying the concepts of Robotics for societal and industrial needs.	**Yes**	Created a robotic surveillance system that addresses critical societal security needs while demonstrating practical applications of advanced robotics concepts in law enforcement and public safety.
PSO 2	To create innovative ideas and solutions for real time problems in Manufacturing sector by adapting the automation tools and technologies.	**Yes**	Developed an innovative surveillance automation solution that can be adapted for security manufacturing environments, perimeter monitoring, and automated inspection systems using real-time AI processing and robotic mobility.